

Dungeon Crawler: Enemy AI Focused Capstone

Project Overview:

This capstone looks to develop a 2D action “rogue-like” game where the player can navigate through an expansive dungeon map to fight a variety of basic enemies and bosses. In these types of games, the player usually attempts the dungeon multiple times since dying will reset all in-game progress, requiring the player to learn the actions and logic of all the enemies and bosses. The game could either be designed from a top-down approach, where the player will be able to see the X and Z plane, or like “metroidvania” games, where they are a side-view of the map and designed to show the X and Y plane instead. This choice is primarily an artistic one, since both types of games still take place within a 2D plane. One of the major differences will be how player/enemy models and the dungeon map will need to be oriented; however, finding/creating art will be a challenge in both cases.

Player Characters:

The player will have access to a variety of characters to play the game as, each with a unique playstyle and some with a focus on abilities that alter the decision-making process of the enemies. The capstone will require:

1. Playable character that does not alter enemy states/behaviors, which can be developed early on
2. Playable character that is directly linked to a state that affects enemy logic
3. Playable character that alters the map to affect enemy pathfinding

If time permits, having a character that changes who the enemies target or try to attack would be a fun challenge. Some examples of these characters may include a gladiator who specializes in melee combat, a pyromancer who can burn enemies, or an ice mage who can freeze the environment.

Enemies and Bosses:

The enemies and bosses the player will be fighting throughout the game will be piloted through the use of finite state machines and behavioral trees to simulate a sense of autonomy. Multiple enemy types will be created for the game, each with a different decision-making model that will be showcased through alternate algorithms, behaviors, and/or states. The game will aim to deliver four unique basic enemy types as well as two bosses: multiple of these basic enemies will be spread throughout the dungeon map that the player will have to fight or avoid, along with bosses being located halfway through the dungeon and at the end, acting as a midpoint and final boss. Each of the basic enemy types will be centered around a different coding challenge that involves their decision-making process. This will include:

1. Basic enemy that can affect the state of the player character
2. Basic enemy that uses alternate pathfinding algorithms
3. Basic enemy that can choose between assisting others or attacking the player
4. Basic enemy that uses an alternate field of view

Some examples of how these could be designed would be a bruiser enemy that can stun the player, an archer enemy that looks for the best vantage point to attack the player, a dark mage who can heal other enemies, and a blind enemy who can't see the player but rather can hear their attacks and movement. The bosses of the game will follow some similar goals, with both bosses needing to affect the state of the player and the map environment. To create some variance between the two bosses, each one should have a unique goal. One goal could be for the boss to have multiple phases, resulting in different state machines or behavioral trees being used for each phase. However, I believe these goals should be created by the individual teams for this capstone, since these could drastically affect the creative aspect of designing a boss fight for the player.

Decision-Making Models:

A combination of both finite state machines and behavioral trees will need to be used for the enemies to operate on their own, along with keeping track of the player's current state. Basic enemies would benefit more from using finite state machines since this will create the most variance in their logic. Many state machines seen within modern games use a mix of states and substates, which act as nested states to simplify the complex logical processes of some enemy AI. Bosses, on the other hand, would benefit more from behavioral trees since this would allow for controlled variance within the boss fights. These behavioral trees can have a mix of root, leaf, composite, and decorator nodes. The root and leaf nodes are similar to those seen in tree data structures, whereas composite and decorator nodes determine the flow of logical processes and modify existing attributes or results. These types of nodes have different subtypes, which can allow for more control over the enemy AI, along with incorporating variance in the actions the enemy AI will take.

Project Goals:

Graphical User Interface:

1. **Game Settings:** Design settings menu for changing controller settings, allowing the user to remap the button layout for the game controls. Along with this will be a main menu that can navigate to a character selection menu, a settings menu, and if time allows, a difficulty settings menu.
2. **Enemy AI Display:** A display that can be accessed through the settings menu while playing, which will monitor a selected enemy's current state machine or behavior tree. A visual representation of the state machines and behavior trees will need to be designed for this portion of the project, along with being able to hover over and examine the different states/behaviors of the targeted enemy. This will also allow the user to manually change the current state/behavior of the enemy for developer testing or player curiosity.

Game Design Goals:

1. **Playable Classes:** Selectable player classes, each with a unique design challenge that affects enemy logic associated with them, along with a base player class that can be developed early on.
2. **Basic Enemy Types:** At least four basic enemy types, each with a unique way of interacting with the player or the environment: such as affecting the player character's state, using different pathfinding algorithms, switching between targeting the player and other enemies, or using alternate algorithms for their field of view.
3. **Boss Fights:** At least two boss fights, each with the ability to alter the player character's state and the environment. Future goals for bosses should be developed based on the vision of the game that the group has.

Backend Integration:

1. **Finite State Machines:** Design state machines for both the player character and enemies, which will keep track of the current state of the player for animations and effects, along with acting as the driver for the enemy AI. This will include multiple states, some unique to different enemy types, and others unique to the player characters. These will dictate what actions can be done by the player or enemy, along with modifying existing algorithms or attributes used by the player or enemy.
2. **Behavioral Trees:** Design behavioral trees for enemies in the game to simulate connected variance within their decision-making process. This can be achieved through using a combination of composite and decorator nodes, along with their respective variations such as sequence, selector, inverter, and repeater nodes.
3. **Algorithms:** Design algorithms, such as the A* algorithm or other greedy search algorithms, for enemy pathfinding. Other algorithms may need to be developed for aspects of enemy logic, such as utility algorithms.

Stretch Goals:

1. **Additional Effects:** Add other effects that alter the NPC logic or map environment, such as a poison that spreads from one NPC to another, resulting in them displacing to avoid spreading the effect.
2. **Additional Characters:** A playable character that can alter the enemies' targeting behavior, potentially by turning them into allies. This may require new algorithms for priority targeting for the enemies that become allies.
3. **Difficulty Features:** Implement an adjustable difficulty slider or selectable presets for the player to adjust before playing the game. This may alter the enemies' pre-defined health, damage, or even how the state machines/behavioral trees operate, depending on how this is chosen to be implemented.
4. **Search Optimizations:** Incorporate optimization techniques into the search algorithms for pathfinding, such as creating a Jump Point Search algorithm or by introducing Goal Bounding into the algorithms.

Art and Technology Considerations:

With the project being based around finite state machines and behavioral trees, many of the preexisting game engines will work, such as the Godot engine, Unity, or Unreal. Each of these also contains access to pre-existing 2D physics engines, whether it be directly built into the engine or there being open-source 2D physics engines such as Box2D. As for the art assets, this may be the most concerning part of the project. While these mentioned game engines do have asset stores with 2D assets for character/enemy models and dungeon maps, these might be out of the budget for some groups, depending on the aesthetic they are aiming for with their game. Thankfully, websites such as CraftPix and Itch.io have free 2D game assets for teams to use. Along with these, more AI tools are becoming available to the public, so using these may be beneficial for groups who are unable to find desirable free assets from already existing online sources. Even though complete sprite sheets for custom 2D characters may be difficult to create with online tools or find, I do believe it can be done within the timeframe of the capstone; though, some groups may need to choose to have simple-looking characters and enemies for the capstone to prevent long turn issues with sprite model animations.